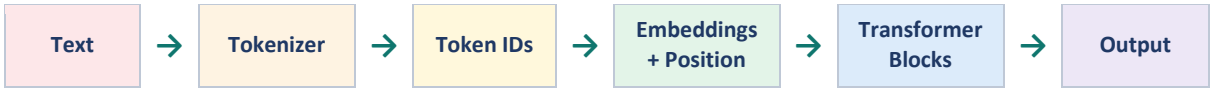
STUDY NOTES

Transformers, GPT-2 and BERT

A clear, illustrated architecture and learning guide

Tokenization · Embeddings · Attention · Add & Norm · Feed-Forward Networks · Training · Text Generation



The one-line pipeline behind every model in these notes.

How to use these notes

These notes consolidate the key concepts into one illustrated reference. Read each architecture diagram **from bottom to top** (inputs at the bottom, output probabilities at the top), then use the comparison tables to see how the original Transformer, GPT-2 and BERT relate to one another.

Learning outcomes

- Explain how text becomes token IDs and vector representations.
- Understand token embeddings, positional information, queries, keys, values and multi-head self-attention.
- Explain residual connections, LayerNorm and the Feed-Forward Network (FFN).
- Trace GPT-2 from input text to next-token prediction and generation.
- Distinguish the original encoder–decoder Transformer, decoder-only GPT-2 and encoder-only BERT.

Roadmap

Part	Topic	Figure / visual
1	The big picture: what is a Transformer?	Figure 1 – three architectures side by side
2	Tokenization and input representations	Token → ID → vector table
3	Self-attention: Q, K and V	Formula box, Q/K/V table
4	Residual connections and LayerNorm	Post-LN vs Pre-LN table
5	Feed-Forward Network (FFN)	GPT-2 Small dimension flow
6	Original Transformer (encoder–decoder)	Figure 2 – original diagram + legend
7	GPT-2 (decoder-only)	Figure 3, causal-mask matrix
8	BERT (encoder-only)	Figure 4, bidirectional matrix
9	Side-by-side comparison	Master comparison table
10–12	Summary, self-test, mental model	Answer key + glossary

1. The big picture: what is a Transformer?

A **Transformer** is a neural-network architecture designed to process sequences using **attention** and **feed-forward layers**. It was introduced in the 2017 paper “Attention Is All You Need.” The original design contains an **encoder stack** and a **decoder stack**. GPT-2 and BERT use the Transformer idea but select different parts of it and use different training objectives.

Core flow



The output depends on the model: **GPT-2** produces scores for next-token prediction; **BERT** produces contextual representations commonly used for classification, token labelling and other understanding tasks.

Figure 1: three architectures at a glance

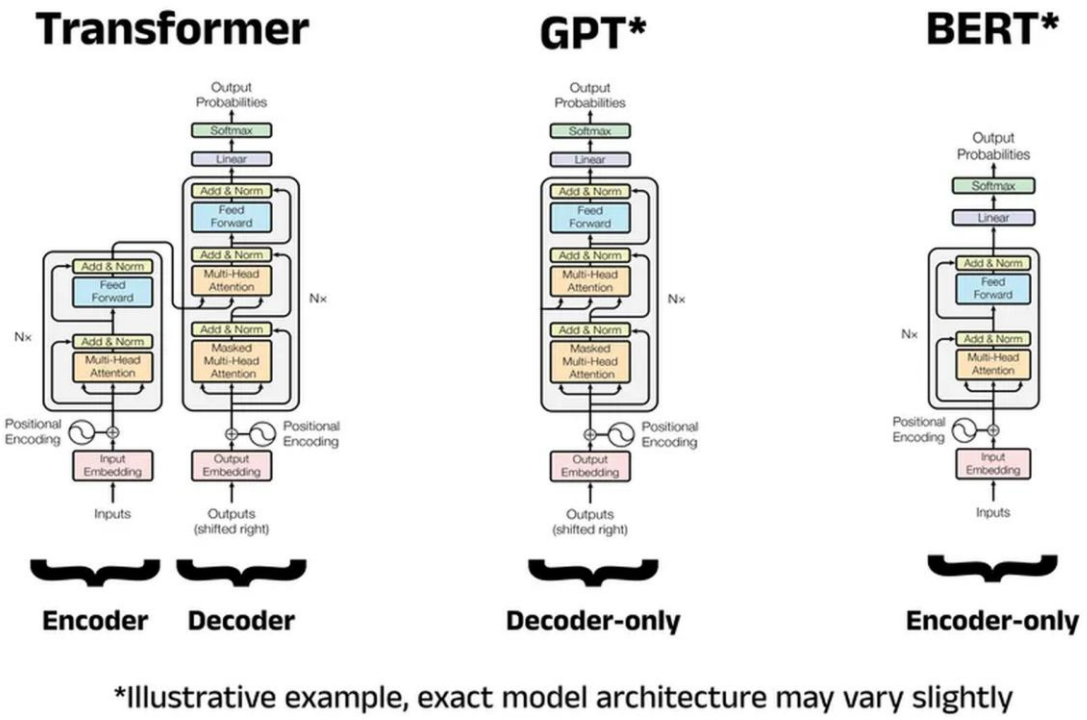


Figure 1. Original Transformer (encoder + decoder), GPT (decoder-only) and BERT (encoder-only). Illustrative; exact architectures may vary slightly.

> **Key idea**
> Same building blocks, different selection. **Transformer** = Encoder + Decoder. **GPT** = Decoder-only. **BERT** = Encoder-only.

2. Tokenization and input representations

2.1 Tokenization

A tokenizer splits text into **tokens** and maps each token to an integer **ID**. A token can be a whole word, a word fragment, punctuation or another text unit. GPT-2 uses **byte-level BPE** tokenization; the original BERT uses **WordPiece**.

Stage	Example (illustrative)	Result
Raw text	“Transformers are useful!”	String
Tokens	[“Transform”, “ers”, “ are”, “ useful”, “!”]	Sub-word pieces
Token IDs	[ID_1, ID_2, ID_3, ID_4, ID_5]	Integers

The exact splits depend on the tokenizer.

2.2 Token embeddings

Each token ID indexes a **learned embedding table**, returning a vector of numbers. These vectors are trainable parameters learned during pretraining.

2.3 Positional information

Attention alone does not inherently encode token order, so the model needs **positional information**.

Model	How position is represented
Original Transformer	Sinusoidal positional encodings (learned encodings discussed as an alternative in the paper)
GPT-2	Learned absolute positional embeddings. Initial representation = token embedding + learned position embedding
BERT (original)	Learned absolute position embeddings, combined with token and segment/type embeddings

> **Remember**
> **Token ID** = an integer label. **Token embedding** = the learned vector looked up using that ID.

3. Self-attention: Q, K and V

Self-attention lets each token build a **contextual representation** by weighting information from permitted tokens. In GPT-2, **causal masking** means a position cannot attend to future positions.

3.1 Query, Key, Value

Item	Intuition	Role
Query (Q)	What information am I looking for?	Represents what the current position is seeking.
Key (K)	What information can I offer?	Used to calculate compatibility with queries.
Value (V)	What information do I contribute?	Content combined according to attention weights.

Q, K and V are calculated from the current input representations using *learned linear projections*. The projection weights are learned during training and remain fixed during ordinary inference; the Q/K/V activations change with the input.

3.2 Scaled dot-product attention

Attention scores are computed from query–key similarities, scaled, masked where necessary, passed through softmax, and used to combine value vectors.

$$\text{Attention}(Q, K, V) = \text{softmax}\left((QK^T / \sqrt{d_k}) + \text{mask} \right) \cdot V$$

Step	What happens
1	Compare each query with every key (QK^T) to get similarity scores.
2	Scale by $\sqrt{d_k}$ to keep scores in a stable range.
3	Add the mask (e.g. block future positions in GPT-2).
4	Apply softmax to turn scores into attention weights.
5	Use the weights to take a weighted sum of the value vectors (V).

Implementations commonly use a **fused QKV projection** and split its output into heads; it is not necessary to implement three separate matrix multiplications.

3.3 Multi-head attention

Multi-head attention uses **several learned attention heads in parallel**. Each head can learn different relationships. The number of heads is a fixed architecture setting, **not** the number of tokens. The head outputs are **concatenated** and projected back to the model's hidden dimension.

4. Residual connections and LayerNorm (Add & Norm)

4.1 Residual connection: "Add"

A residual connection adds a sublayer's **input to its output**, element by element. Dimensions must match. Residual paths help preserve information and support gradient flow through deep networks.

Input X	Sublayer output A	X + A	Note
[2, 4]	[1, 3]	[3, 7]	Element-wise addition

4.2 Layer Normalization: "Norm"

LayerNorm normalizes the features **within each token representation** using that vector's mean and variance. It includes a small epsilon for numerical stability and learned **scale (gamma)** and **shift (beta)** parameters. LayerNorm does **not** mix information across token positions.

	Original Transformer	GPT-2
Style	Post-LayerNorm (as in the published block diagram)	Pre-LayerNorm
Order	Sublayer → add residual → normalize	Normalize → sublayer → add to residual stream
Final norm	—	Extra final LayerNorm after the stack

5. Feed-Forward Network (FFN)

Attention mixes information across token positions; the FFN transforms features within each position. In a standard Transformer FFN, the same learned network is applied independently to each token vector, typically in parallel.

$$\mathbf{FFN(x) = W_2 \cdot GELU(\ W_1 x + b_1\ ) + b_2}$$

GPT-2 Small dimensions

Hidden 768 → Linear (expand) → 3072 + GELU → Linear (project) → Hidden 768

The expansion creates a wider intermediate feature space; **GELU** adds nonlinearity; the second projection returns to the hidden dimension so the FFN output can be added through the residual connection. These dimensions are specific to **GPT-2 Small**.

6. The original Transformer architecture

The original Transformer is **encoder–decoder**. The encoder processes the input sequence. The decoder generates the target sequence and uses both **causal self-attention** and **encoder–decoder cross-attention**.

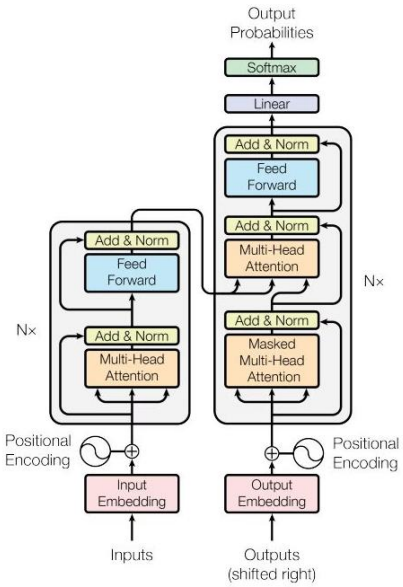


Figure 2. The Transformer – model architecture (Vaswani et al., 2017).

How to read Figure 2

- **Left stack = Encoder** (N× identical blocks).
- **Right stack = Decoder** (N× identical blocks).
- Pink boxes: embeddings. The ⊕ circle adds **positional encoding**.
- Orange boxes: attention. **Masked** attention hides future tokens.
- Blue boxes: feed-forward networks.
- Yellow bars: Add & Norm (residual + LayerNorm).
- The arrow from the encoder into the decoder's middle attention block is **cross-attention**.
- Top: Linear → Softmax → output probabilities.

6.1 Encoder block

1. Input embeddings + positional encoding
2. Multi-head self-attention (positions can generally attend to all unmasked input positions)
3. Residual connection + LayerNorm

4. Position-wise feed-forward network
5. Residual connection + LayerNorm

6.2 Decoder block

1. Target embeddings shifted right + positional encoding
2. Masked multi-head self-attention
3. Residual connection + LayerNorm
4. Cross-attention: decoder queries attend to encoder outputs as keys/values
5. Residual connection + LayerNorm
6. Position-wise feed-forward network
7. Residual connection + LayerNorm

The original Transformer uses the decoder to generate outputs **autoregressively**. Its cross-attention is an important difference from GPT-2, which has no separate encoder stack.

7. GPT-2 architecture (decoder-only)

GPT-2 uses a stack of **decoder-style Transformer blocks**, without the original encoder or encoder–decoder cross-attention. It is pretrained with a **causal next-token prediction** objective.

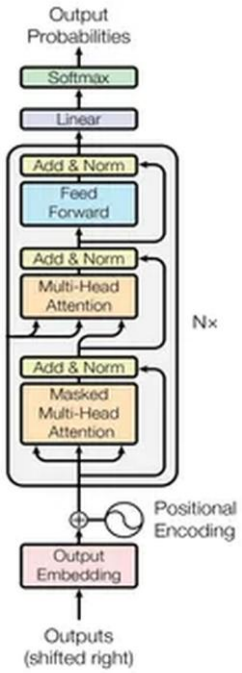


Figure 3. GPT (decoder-only), illustrative.

How to read Figure 3

- Only the **decoder side** of the original design remains: no encoder, no encoder input.
- Input is “Outputs (shifted right)”: the text so far.
- Output probabilities are over the **next token**.

> **Reading the diagram correctly**
>
> Figure 3 is **illustrative**. It keeps a second Multi-Head Attention block from the original decoder drawing. In actual GPT-2, each block has **one causal self-attention sublayer and one FFN, with no cross-attention**.
>
> The diagram also shows Post-LN "Add & Norm" labels; real GPT-2 uses **Pre-LayerNorm** plus a final LayerNorm.

7.1 GPT-2 end-to-end flow

1. Text → GPT-2 tokenizer → token IDs.
2. Token embeddings + learned absolute positional embeddings.
3. Repeated GPT-2 blocks: Pre-LayerNorm → causal multi-head self-attention → residual addition → Pre-LayerNorm → FFN (linear → GELU → linear) → residual addition.
4. Final LayerNorm.
5. Linear language-model head produces vocabulary **logits** at each sequence position.
6. Softmax can interpret logits as probabilities; a decoding strategy selects the next token.
7. Append the selected token and repeat until a stopping condition or context limit is reached.

7.2 Causal masking

A token position can attend to itself and earlier positions, but **not future positions**. This prevents information leakage during training and matches autoregressive generation.

Causal (GPT-2) attention pattern, 4-token example

Query ↓ / Key →	Tok 1	Tok 2	Tok 3	Tok 4
Tok 1	✓ can see	✗ masked	✗ masked	✗ masked
Tok 2	✓ can see	✓ can see	✗ masked	✗ masked
Tok 3	✓ can see	✓ can see	✓ can see	✗ masked
Tok 4	✓ can see	✓ can see	✓ can see	✓ can see

During **training**, shifted targets allow predictions at many positions to be calculated **in parallel**. During **generation**, newly generated tokens are fed back **sequentially**.

7.3 Training objective

GPT-2 predicts the next token and is trained with a **cross-entropy loss** over the correct target tokens. Backpropagation computes gradients, and an optimizer updates the learned parameters. At ordinary inference time, the parameters are **not** updated.

7.4 Logits and decoding

Logits are raw vocabulary scores. **Softmax** turns them into a probability distribution. These decoding options are choices, not separate model architectures:

Decoding choice	What it does
Greedy decoding	Chooses the highest-probability token.

Decoding choice	What it does
Sampling	Chooses probabilistically from the distribution.
Temperature	Changes the sharpness of the distribution.
Top-k sampling	Restricts sampling to the k highest-scoring candidates.

8. BERT architecture (encoder-only)

BERT stands for **Bidirectional Encoder Representations from Transformers**. It uses a stack of Transformer **encoder** blocks. Its self-attention is **bidirectional**: a token can attend to both left and right context, subject to padding or other masks.

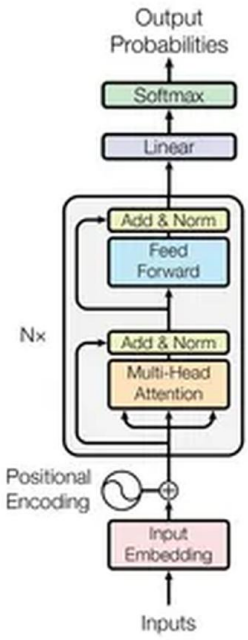


Figure 4. BERT (encoder-only), illustrative.

How to read Figure 4

- Only the **encoder side** remains: Input Embedding → N× [Multi-Head Attention → Add & Norm → Feed Forward → Add & Norm].
- No masked attention: every token sees the whole sequence.
- No decoder and no cross-attention.
- The output head (Linear + Softmax) depends on the task, e.g. predicting masked tokens.

8.1 BERT input representations

In the original BERT design, the input combines **token embeddings**, **learned positional embeddings** and **segment/token-type embeddings** (to distinguish sentence A and sentence B). It uses WordPiece tokenization.

8.2 BERT block

1. Input representations
2. Multi-head bidirectional self-attention
3. Residual connection + LayerNorm
4. Feed-forward network
5. Residual connection + LayerNorm

6. Repeat for the encoder stack

Bidirectional (BERT) attention pattern, 4-token example

Query ↓ / Key →	Tok 1	Tok 2	Tok 3	Tok 4
Tok 1	✓ can see	✓ can see	✓ can see	✓ can see
Tok 2	✓ can see	✓ can see	✓ can see	✓ can see
Tok 3	✓ can see	✓ can see	✓ can see	✓ can see
Tok 4	✓ can see	✓ can see	✓ can see	✓ can see

Shown without padding or other masks, which would block some positions.

8.3 Original BERT pretraining objectives

Objective	Description
Masked Language Modeling (MLM)	Selected input tokens are masked or otherwise corrupted; the model predicts their original identities using surrounding context.
Next Sentence Prediction (NSP)	Binary task in the original setup: does the second segment follow the first in the source text? Later BERT-style models may omit or replace NSP.

> **Common mistake**
>
> BERT is **not** a standard left-to-right autoregressive generator like GPT-2. Its contextual outputs are used for classification, named-entity recognition, extractive question answering and similar understanding tasks.

9. Original Transformer vs GPT-2 vs BERT

Feature	Original Transformer	GPT-2	BERT
Overall architecture	Encoder–decoder	Decoder-only	Encoder-only
Attention pattern	Encoder self-attention; decoder causal self-attention + cross-attention	Causal self-attention	Bidirectional self-attention
Main original training / use	Sequence-to-sequence, e.g. translation	Autoregressive next-token prediction	Masked language modeling; original setup also NSP
Future tokens visible?	Encoder: generally yes; decoder self-attention: no	No	Yes, within the input, except masked/padded positions
Typical strengths	Input-to-output sequence transformation	Text generation	Contextual text understanding

Feature	Original Transformer	GPT-2	BERT
Position information	Sinusoidal in the original paper (learned option discussed)	Learned absolute positional embeddings	Learned absolute positional embeddings
Cross-attention	Yes, in the decoder	No separate encoder, so none	No encoder–decoder cross-attention

10. One-page summary to remember

- **Tokenization** converts text into token IDs; **embeddings** map IDs to vectors.
- **Positional information** lets the model represent order.
- **Q/K/V** support attention: queries match keys; weights combine values.
- **Multi-head attention** runs multiple learned heads in parallel.
- **Residual connections** add the sublayer input to its output; **LayerNorm** normalizes features.
- **FFN** applies learned nonlinear transformations independently at each token position.
- Original Transformer = **encoder + decoder**.
- GPT-2 = **decoder-only + causal attention + next-token prediction**.
- BERT = **encoder-only + bidirectional attention + masked language modeling**.
- GPT-2 loop: logits → softmax/probabilities → decoding choice → next token → repeat.

11. Self-test questions

Try answering each question before checking the answer key in Appendix A.

1. What is the difference between a token ID and a token embedding?
2. Why does a Transformer need positional information?
3. What roles do queries, keys and values play in attention?
4. Why does GPT-2 use causal masking?
5. What is the difference between attention and the FFN?
6. What does a residual connection do, and what does LayerNorm do?
7. What does GPT-2's output projection produce?
8. How are GPT-2 and BERT different in architecture and training objective?
9. Why can GPT-2 train predictions at many positions in parallel but generate tokens sequentially?

12. Recommended mental model

Think of the Transformer as a **repeated information-processing pipeline**. Embeddings represent tokens; attention lets positions exchange context; the FFN transforms each position's features; residual connections and LayerNorm support stable deep processing. The **architecture and attention mask** determine what context is available, while the **training objective** determines what the model learns to predict.

> **Terminology note**
> "Add & Norm" is often shown as a combined label in the original Transformer diagram. For precise GPT-2 architecture, remember that GPT-2 uses **Pre-LayerNorm** rather than copying the original diagram's Post-LayerNorm ordering.

Appendix A: Answer key

1. A token ID is the integer the tokenizer assigns to a token. A token embedding is the learned vector looked up in the embedding table using that ID.
2. Attention by itself does not encode token order, so positional information is added so the model can represent where each token sits in the sequence.
3. Queries describe what a position is looking for, keys describe what each position offers (matched against queries to give weights), and values carry the content that is combined using those weights.
4. Causal masking stops a position from seeing future tokens. This prevents information leakage in training and matches autoregressive generation.
5. Attention mixes information across token positions; the FFN transforms the features within each position independently.
6. A residual connection adds a sublayer's input to its output, preserving information and helping gradient flow. LayerNorm normalizes the features within each token vector (it does not mix positions).
7. A vector of logits over the vocabulary at each sequence position; softmax converts them to probabilities.
8. GPT-2 is decoder-only with causal attention and is trained for next-token prediction. BERT is encoder-only with bidirectional attention and is trained with masked language modeling (plus NSP in the original).
9. In training the full target sequence is known, so with causal masking and shifted targets all positions are predicted at once. In generation each new token depends on the previously chosen one, so tokens must be produced one after another.

Appendix B: Quick glossary

Term	Meaning
Token	A text unit (word, word piece, punctuation) produced by the tokenizer.
Embedding	A learned vector representing a token.
Self-attention	Mechanism where each position weights information from other permitted positions.
Causal mask	Mask that blocks attention to future positions.
Residual connection	Adding a sublayer's input to its output.
LayerNorm	Normalization of features within one token vector.
FFN	Position-wise feed-forward network (linear → GELU → linear).
Logits	Raw vocabulary scores before softmax.
Autoregressive	Generating one token at a time, feeding each back as input.
MLM / NSP	BERT's masked-token prediction and next-sentence prediction objectives.